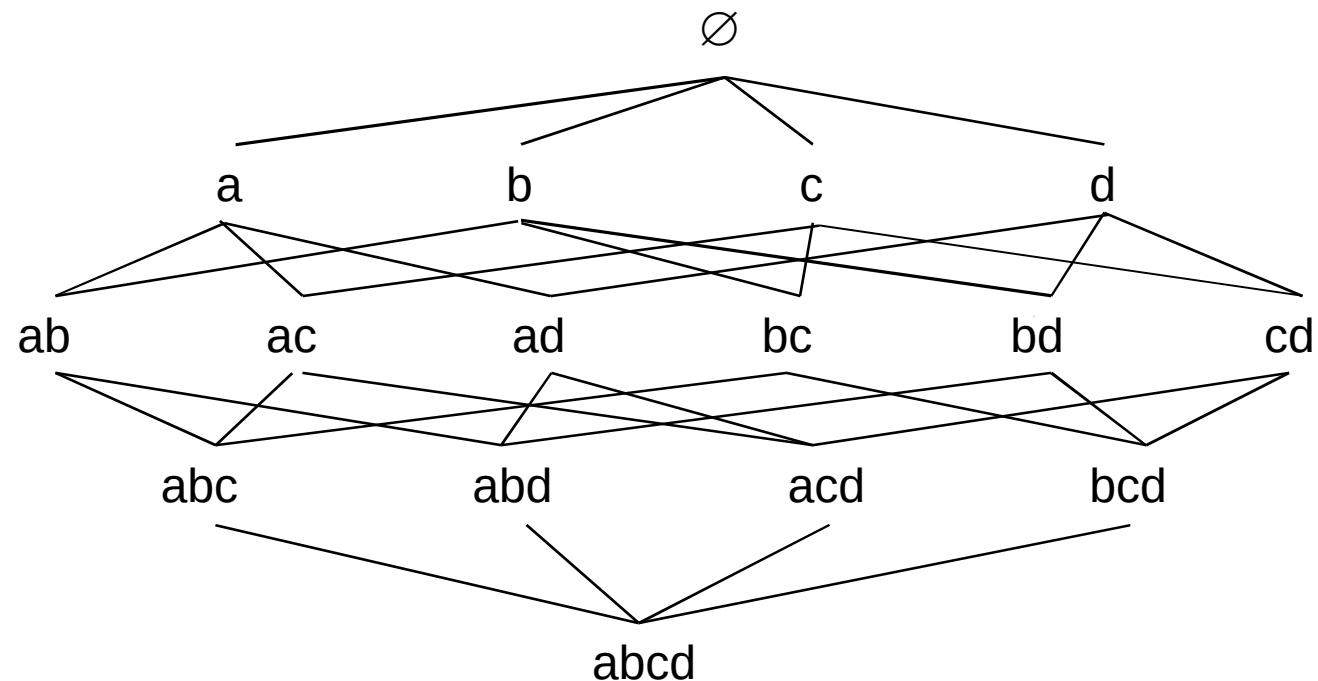


Finding Frequent Itemsets

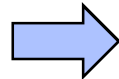
- ▶ Objective: given a dataset, find all the itemsets (or sets of attribute-value pairs) that satisfy the minimum support count.
- ▶ Algorithms
 - ▶ Apriori
 - ▶ FP-Growth
 - ▶ H-Mine
 - ▶ Eclat
 - ▶ Partition
 - ▶ CLOSET
 - ▶ CHARM
 - ▶ etc.

Search Space for Finding All Frequent Itemsets

- ▶ Search space for D with 4 items:

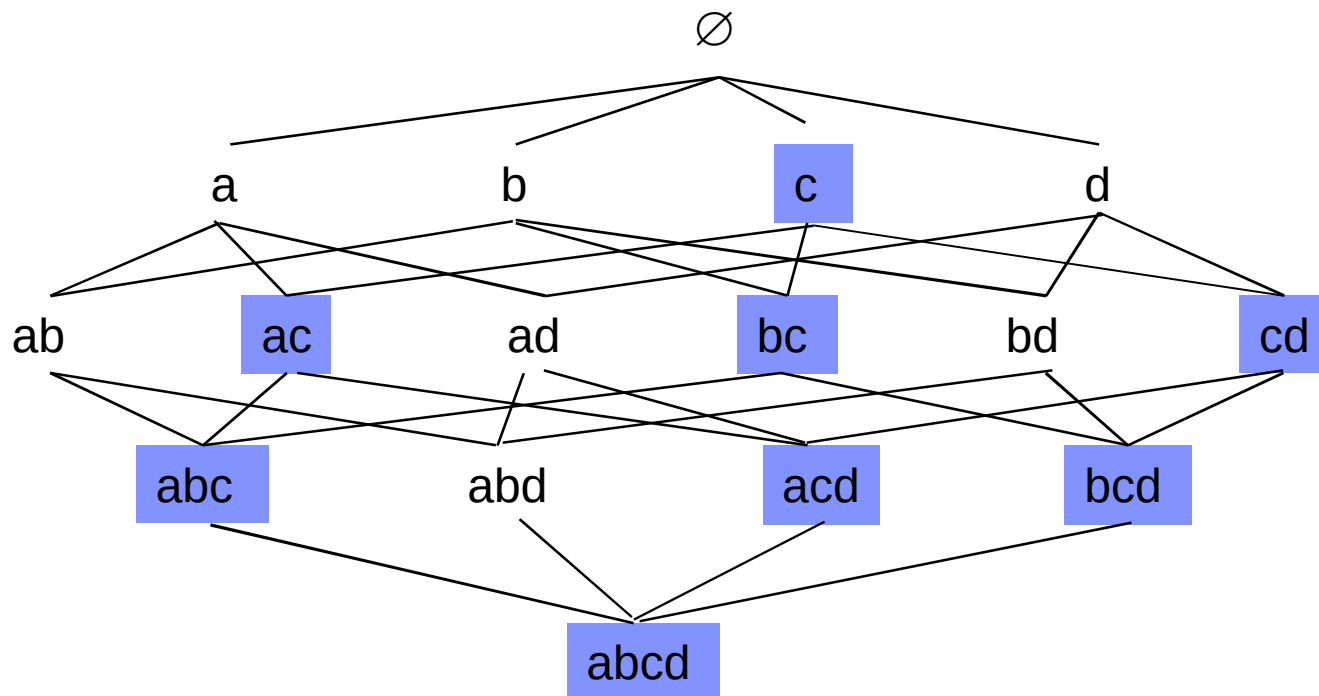


Apriori Property

- ▶ The **Apriori** property (an *anti-monotone* property), also called *downward closure* property:
 - Any nonempty subset of a frequent itemset must be frequent
 - ▶ i.e., if $\{A,B,C\}$ is a frequent itemset, $\{A,B\}$, $\{A,C\}$, $\{B,C\}$, $\{A\}$, $\{B\}$ and $\{C\}$ must also be frequent.
 - ▶ This is because a transaction containing $\{A, B, C\}$ also contains $\{A,B\}$, $\{B,C\}$, Thus, the support count of a subset is not less than that of the superset.
- ▶ Another way to express the property:
 - ▶ If an itemset is infrequent, none of its superset is frequent.
- ▶ No superset of any infrequent itemset should be checked.
 - ▶ Many item combinations can be pruned from the search space. 

Pruning Search Space Using Apriori Property

- If c is not frequent, all the sets containing c are pruned:



Apriori Algorithm

- ▶ Level-wise iterative generation
 - ▶ Generate length $(k+1)$ candidate itemsets from frequent k -itemsets
 - ▶ Test the candidates against dataset D

The Apriori Algorithm

- ▶ Based on the Apriori property, use *iterative level-wise approach* and *candidate generation-and-test*

- ▶ Pseudo-code:

C_k : a set of candidate itemsets of size k

L_k : the set of frequent itemsets of size k

$L_1 = \{\text{frequent items}\};$ (*Scan dataset to find all frequent 1-itemsets*)

for ($k = 1; L_k \neq \emptyset; k++$) **do begin**

$C_{k+1} = \underline{\text{candidates generated from } L_k};$

if $C_{k+1} \neq \emptyset$

for each transaction t in dataset **do**

 increment the count of all candidates in C_{k+1}
 that are contained in t

} *Scan dataset to
calculate support
for each itemset in
 C_{k+1}*

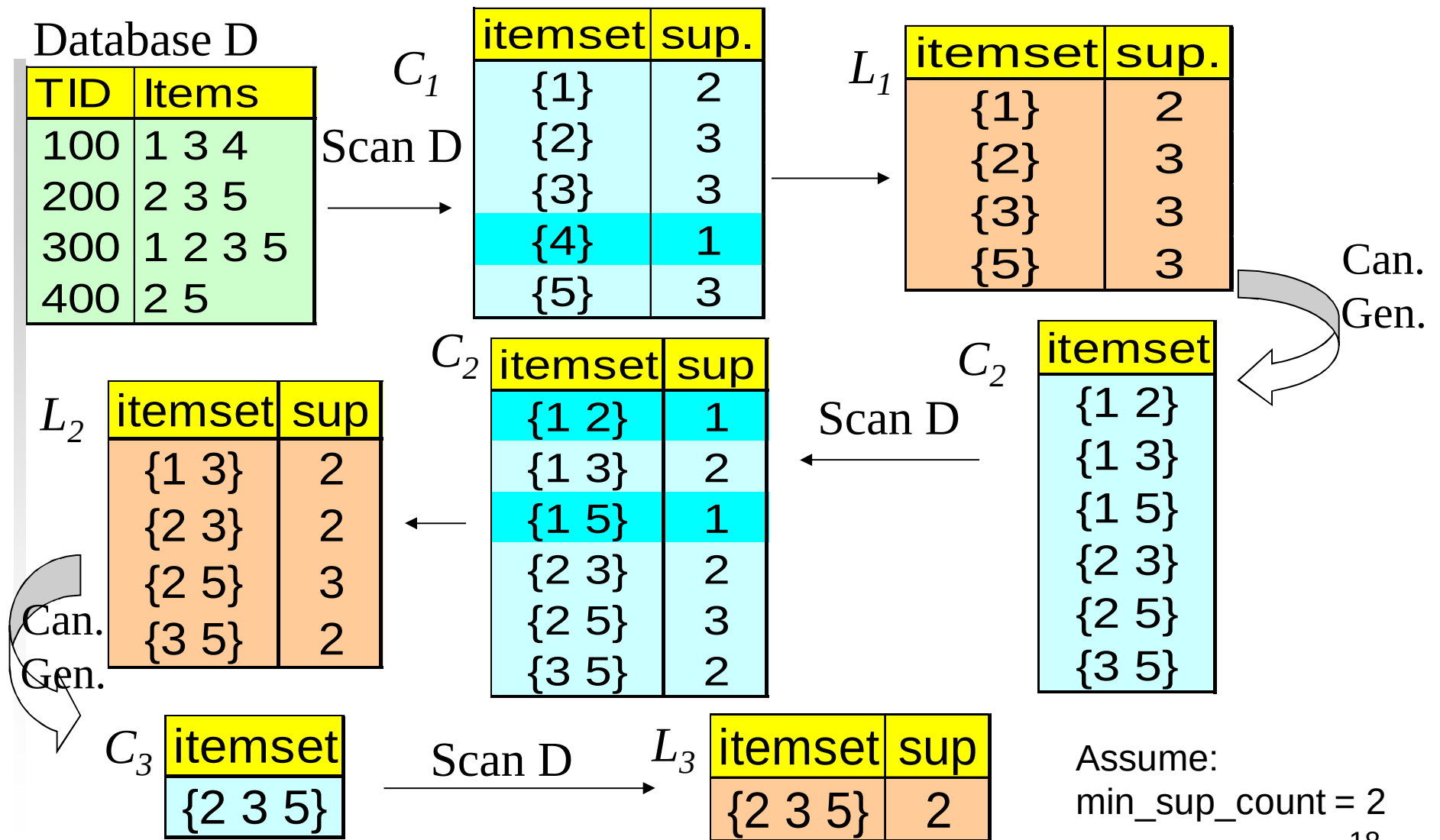
$L_{k+1} = \text{candidates in } C_{k+1} \text{ satisfying min_support}$

end

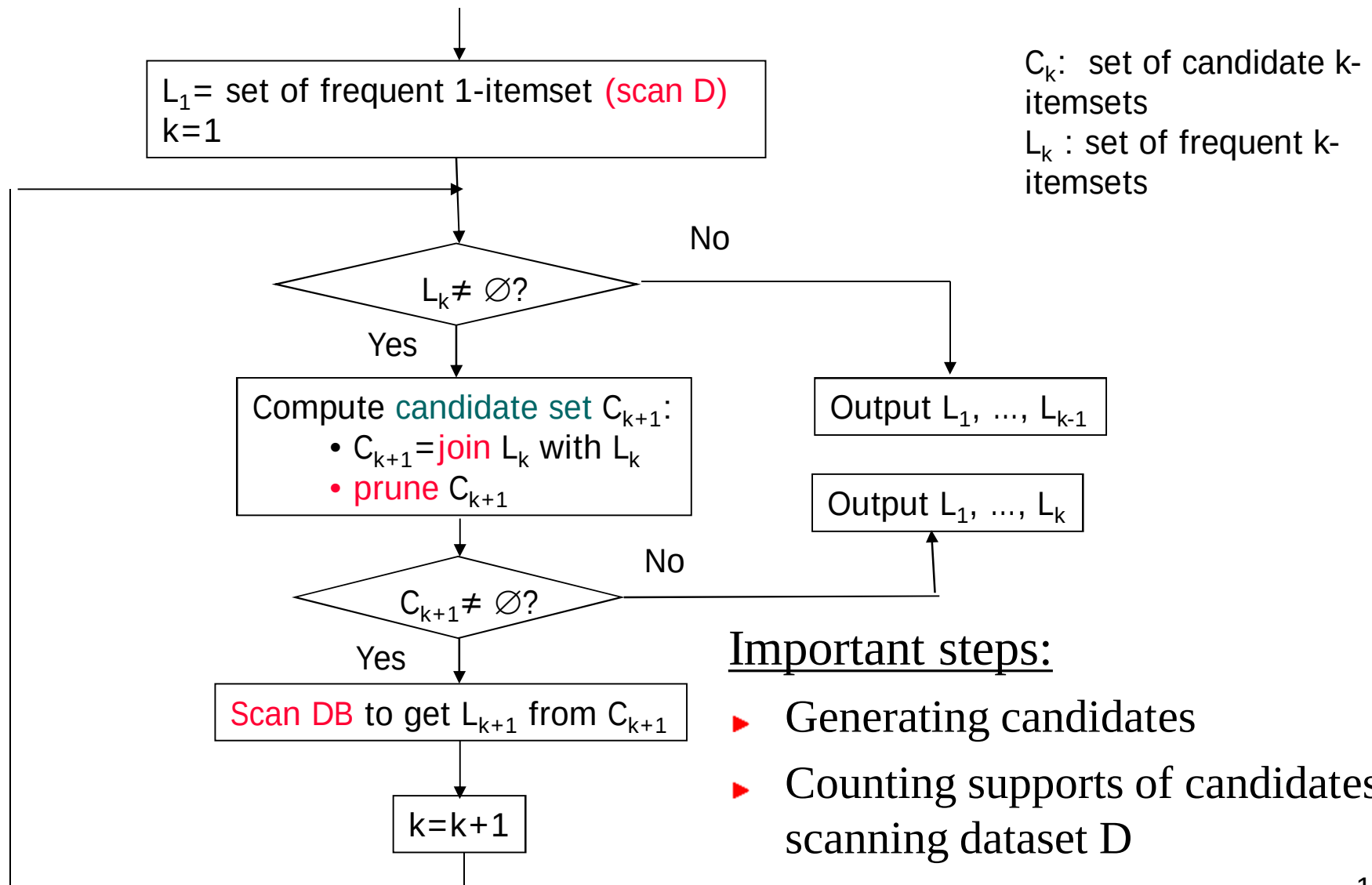
return $\cup_k L_k$;

Level-wise Generation process: $L_k \rightarrow C_{k+1} \rightarrow L_{k+1}$

The Apriori Algorithm — Example



Apriori Algorithm (Flow Chart)



How to Generate Candidates (Generate C_2 from L_1)

- ▶ Get all the two-item combinations from L_1
- ▶ Example:
 - ▶ $L_1 = \{1, 2, 3, 5\}$
 - ▶ $C_2 = \{\{1\ 2\}, \{1\ 3\}, \{1\ 5\}, \{2\ 3\}, \{2\ 5\}, \{3\ 5\}\}$

How to Generate Candidates?

(i.e., How to Generate C_{k+1} from L_k when $k \geq 2$)

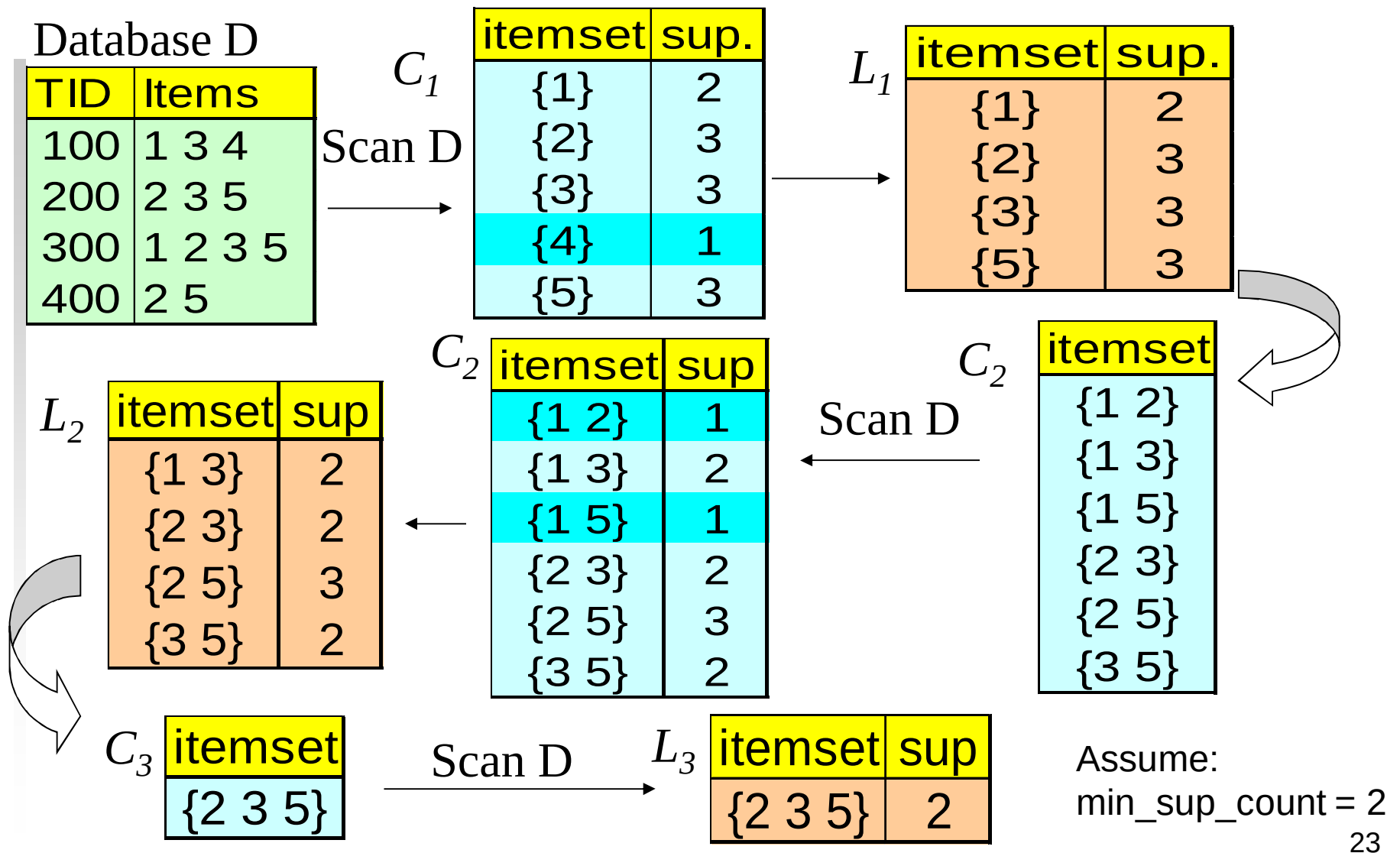
- ▶ Given L_k = the set of frequent k -itemsets
- ▶ List the items in each itemset of L_k in an order
 $L_3 =$

{1 2 3}
{1 2 4}
{1 3 4}
{1 3 5}
{2 3 4}
- ▶ Given L_k , generate C_{k+1} in two steps:
 - ▶ **Join Step**: Join L_k with L_k by joining two k -itemsets in L_k . Two k -itemsets are joinable if their first (k-1) items are the same and the last item in the first itemset is smaller than the last item in the second itemset (the condition for joining two members of L_k).
Now, $C_4 = \{\{1\ 2\ 3\ 4\}, \{1\ 3\ 4\ 5\}\}$
 - ▶ **Prune Step**: Delete all candidates in C_{k+1} that have a non-frequent subset by checking all length- k subsets of a candidate
 - ▶ Now, $C_4 = \{\{1\ 2\ 3\ 4\}\}$

Example of Candidate-generation

- ▶ $L_4 = \{abcd, abcg, abdg, abef, abeh, acdg, bcdg\}$
- ▶ Self-joining: $L_4 * L_4$
 - ▶ $abcdg$ from $abcd$ and $abcg$
 - ▶ $abefh$ from $abef$ and $abeh$
- ▶ Pruning:
 - ▶ $abefh$ is removed because $abfh$ or $aefh$ or $befh$ is not in L_4
- ▶ $C_5 = \{abcdg\}$

The Apriori Algorithm — Example

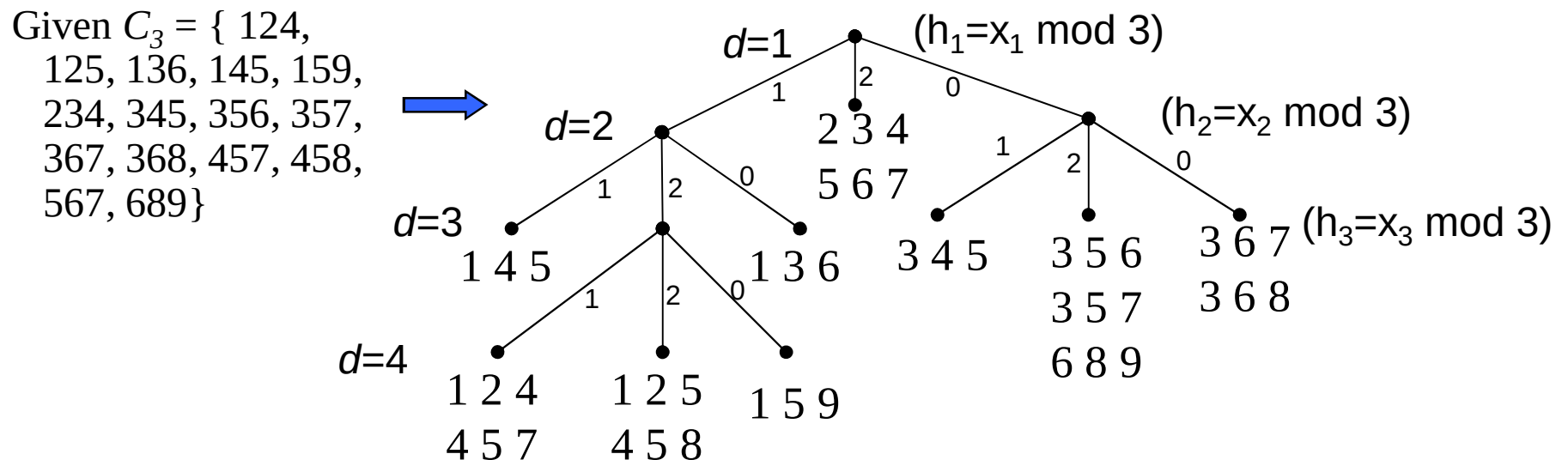


Support Counting of Candidates in dataset (DS) scan

- ▶ Objective of candidate support counting
 - ▶ Find frequent itemsets L_k from a set of candidates C_k
- ▶ **Naïve method**: match each candidate with each transaction. Time consuming when
 - ▶ the total number of candidates is very large
 - ▶ one transaction contains many candidates
- ▶ **Hash tree method**:
 - ▶ Store candidate itemsets in C_k in a hash-tree
 - ▶ Leaf node of hash-tree contains a list of candidates and their counts
 - ▶ Interior node represents a hash function.
 - ▶ Use a subset function to find all the candidates contained in a transaction

Building a Hash Tree

- Initially treat the root as a leaf node
- Insert itemsets in C_k into the tree
- When the number of itemsets in a leaf node exceeds a *specified threshold* (such as 3 in the following example), convert the leaf node into an interior node by
 - hashing the itemsets according to the d -th item of the itemset, where d is the level of the node.

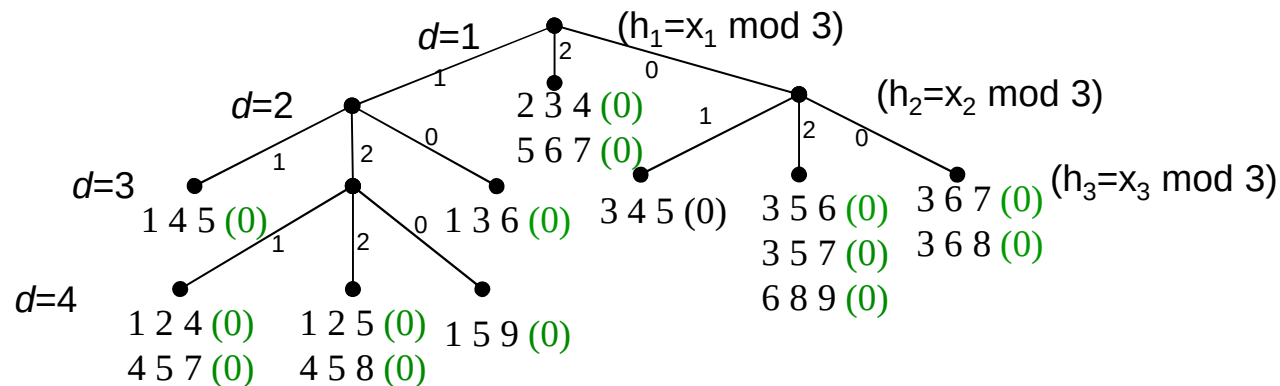


Scan DS to Obtain the Counts of All Length-3 Candidates (1)

- Assume a transaction DB:

$\{1,3,5,6\}$
 $\{2,3,5\}$
 $\{1,3,6,7,8\}$
 $\{3,5,6\}$
.....

- The counts of all the candidates in the hash tree are initialized to zero.



Scan DB to Obtain the Counts of All Length-3 Candidates (2)

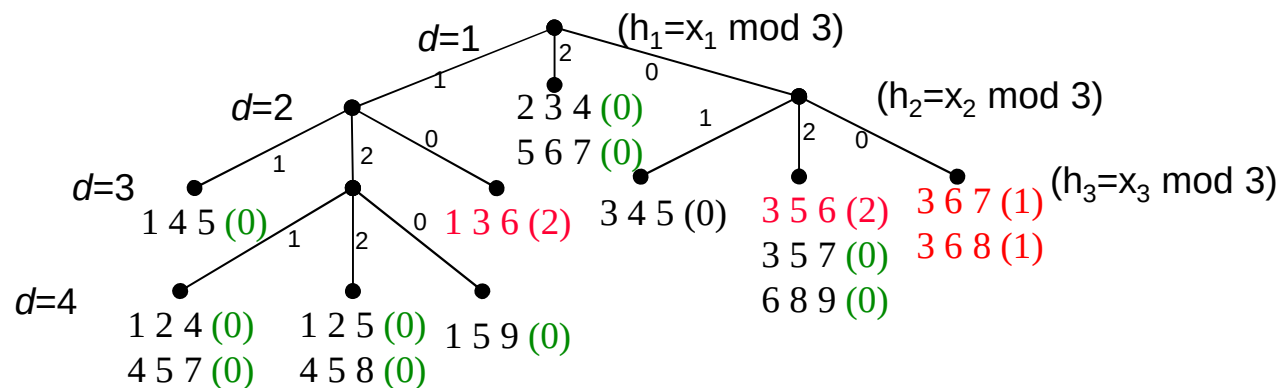
- Assume a transaction DB:

{1,3,5,6}
 {2,3,5}
 {1,3,6,7,8}
 {3,5,6}

General idea: for each transaction,

- Get all the length-3 subsets
- Check each subset to see if it is in the hash tree
- If yes, increase the count of the subset in the tree.

- Apply the **subset function** on each transaction and increase the count of a candidate if the transaction contains the candidate



Subset Function

▶ Functionality

- ▶ Given C_k (in a hash tree) and a transaction t , find all the candidates in C_k contained in t and increase the count of these candidates:

$\text{Subset}(C_k, t)$: candidate itemsets contained in t

▶ Method

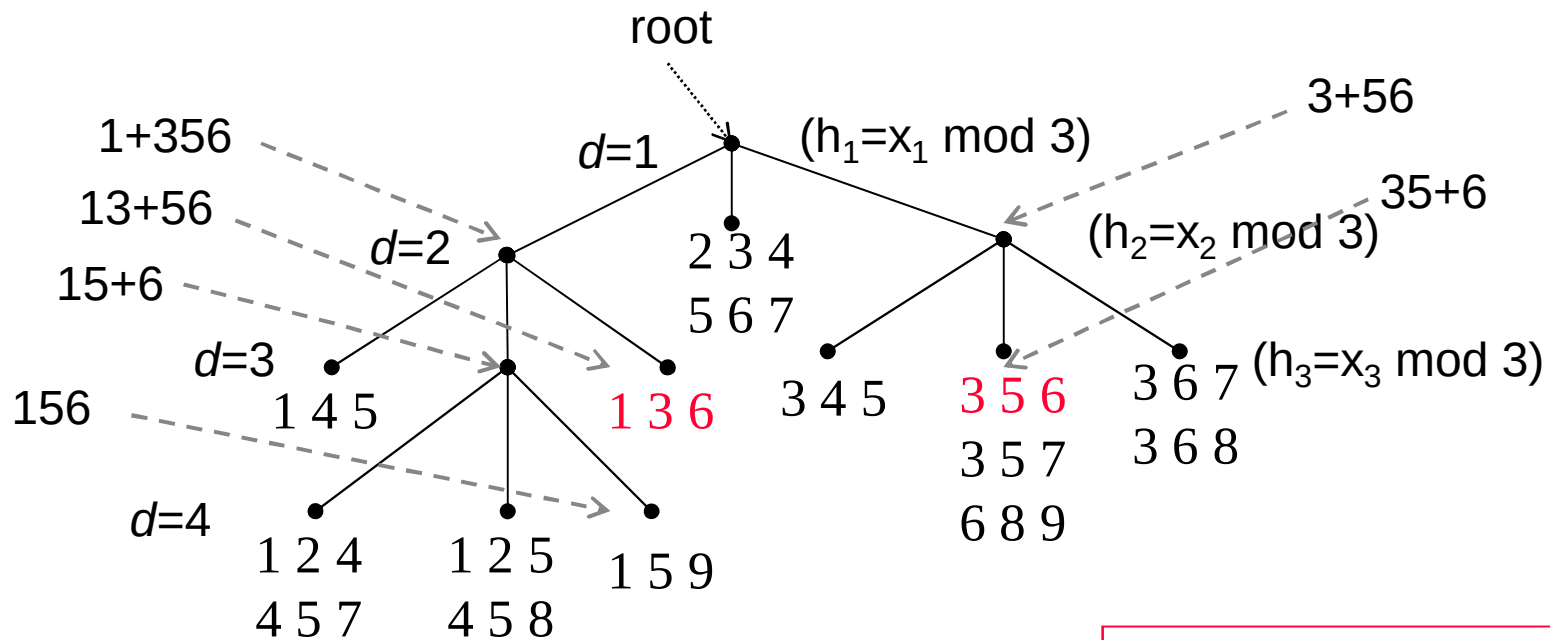
- ▶ Try to check each subset of t to see if it is contained in the tree as follows:
- ▶ At root, hash on every item in t (until the k th item from the end) .
- ▶ At an interior node reached by hashing on item i , hash on each item that comes after i in t (until the $(k-d+1)$ th item from the end, where d is the level of the node in the tree), recursively apply to the nodes in the corresponding bucket
- ▶ At a leaf, find itemsets contained in t

▶ Benefit

- ▶ Don't have to match each candidate with each transaction.

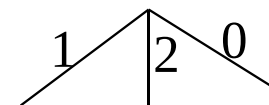
Example: finding the candidates contained in a transaction.

- ▶ Given a transaction {1 3 5 6} and hash tree:



It only goes to the leaf that may contain subsets of the transaction.

hash function



How to Mine Association Rules

- ▶ A two-step process:
 - ▶ *Find all frequent itemsets* ---- *done*
 - ▶ *Generate strong association rules from frequent itemsets.*
- ▶ Example: given min_sup=50% and min_conf=50%

Transaction ID	Items Bought
2000	A,B,C
1000	A,C
4000	A,D
5000	B,E,F



Frequent Itemset	Support
{A}	75%
{B}	50%
{C}	50%
{A, C}	50%

- ▶ Generate strong rules:
 - ▶ {A} → {C} [support=50%, confidence=66.6%]
 - ▶ {C} → {A} [support=50%, confidence=100%]

Generate Association Rules from Frequent Itemsets

► Naïve algorithm:

```
for each frequent itemset  $l$  whose length  $\geq 2$  do
  for each nonempty proper subset  $s$  of  $l$  do
    if (support( $l$ )/support( $s$ )  $\geq$  min_conf)
      output the rule  $s \rightarrow l-s$ , with
        support = support( $l$ ) and
        confidence = support( $l$ )/support( $s$ )
```

Note that we only need to check the confidence.
Do we need to scan the database? No. Why?

Generate Association Rules from Frequent Itemsets

► Example:

- Given a frequent itemset: $l = \{A, B, C\}$
- nonempty proper subsets of l are $\{A, B\}$, $\{A, C\}$, $\{B, C\}$, $\{A\}$, $\{B\}$, $\{C\}$
- resulting association rules:

$$\{A, B\} \rightarrow \{C\}, \text{ confidence} = \frac{\text{support}(\{A, B, C\})}{\text{support}(\{A, B\})} = 50\%$$

$$\{A, C\} \rightarrow \{B\}, \text{ confidence} = \frac{\text{support}(\{A, B, C\})}{\text{support}(\{A, C\})} = 100\%$$

$$\{B, C\} \rightarrow \{A\}, \text{ confidence} = \frac{\text{support}(\{A, B, C\})}{\text{support}(\{B, C\})} = 100\%$$

$$\{A\} \rightarrow \{B, C\}, \text{ confidence} = \frac{\text{support}(\{A, B, C\})}{\text{support}(\{A\})} = 33\%$$

$$\{B\} \rightarrow \{A, C\}, \text{ confidence} = \frac{\text{support}(\{A, B, C\})}{\text{support}(\{B\})} = 29\%$$

$$\{C\} \rightarrow \{A, B\}, \text{ confidence} = \frac{\text{support}(\{A, B, C\})}{\text{support}(\{C\})} = 100\%$$

These confidence values are make-up numbers. This example is not continued previous examples.

If minimum confidence threshold is 70%, only 3 rules are outputted.

Is Apriori Fast Enough? — Performance Bottlenecks

- ▶ The core of the Apriori algorithm:
 - ▶ Use frequent k -itemsets to generate candidate $(k+1)$ -itemsets
 - ▶ Use dataset scan and (hash-tree facilitated) pattern matching to collect counts for the candidate itemsets to generate frequent $(k+1)$ -itemsets from $k+1$ candidate set
- ▶ The bottleneck of *Apriori*: Candidate generation and testing
 - ▶ Huge candidate sets:
 - ▶ 10^4 frequent 1-itemsets will generate more than 10^7 candidate 2-itemsets
 - ▶ To discover a frequent pattern of size 100, e.g., $\{a_1, a_2, \dots, a_{100}\}$, one needs to generate at least $2^{100} \approx 10^{30}$ candidates.
 - ▶ Multiple scans of database:
 - ▶ Needs n or $n+1$ scans, n is the length of the longest frequent pattern

Outline

- ▶ Basic concepts of association rule learning
- ▶ Apriori algorithm
- ▶ *FP-Growth Algorithm*
 - ▶ *Avoid candidate generation*
 - ▶ *Reduce the number of database scans*
- ▶ Finding interesting rules.

FP-Growth

- ▶ J. Han, J. Pei, and Y. Yin. *Mining Frequent Patterns without Candidate Generation.*, Proc. 2000 ACM-SIGMOD Int. Conf. on Management of Data (SIGMOD'00), Dallas, TX, May 2000.
- ▶ J. Han, J. Pei, Y. Yin and R. Mao, *Mining Frequent Patterns without Candidate Generation: A Frequent-Pattern Tree Approach*, Data Mining and Knowledge Discovery, 8(1):53-87, 2004. (http://www-faculty.cs.uiuc.edu/~hanj/pdf/dami04_fptree.pdf)
- ▶ Chapter 6.2.4 (3rd edition) or Chapter 5.2.4 (2nd Edition)

Mining Frequent Patterns Without Candidate Generation (FP-growth)

Two major steps:

- ▶ Compress a large database into a compact, Frequent-
Pattern tree (FP-tree) structure
 - ▶ highly condensed, but complete for frequent pattern mining
 - ▶ **avoid costly database scans**
- ▶ Mine frequent patterns (itemsets) from an FP-tree
 - ▶ A divide-and-conquer methodology: decompose mining tasks into smaller ones
 - ▶ Efficient: **avoid candidate generation** -- generate frequent patterns from the tree directly.

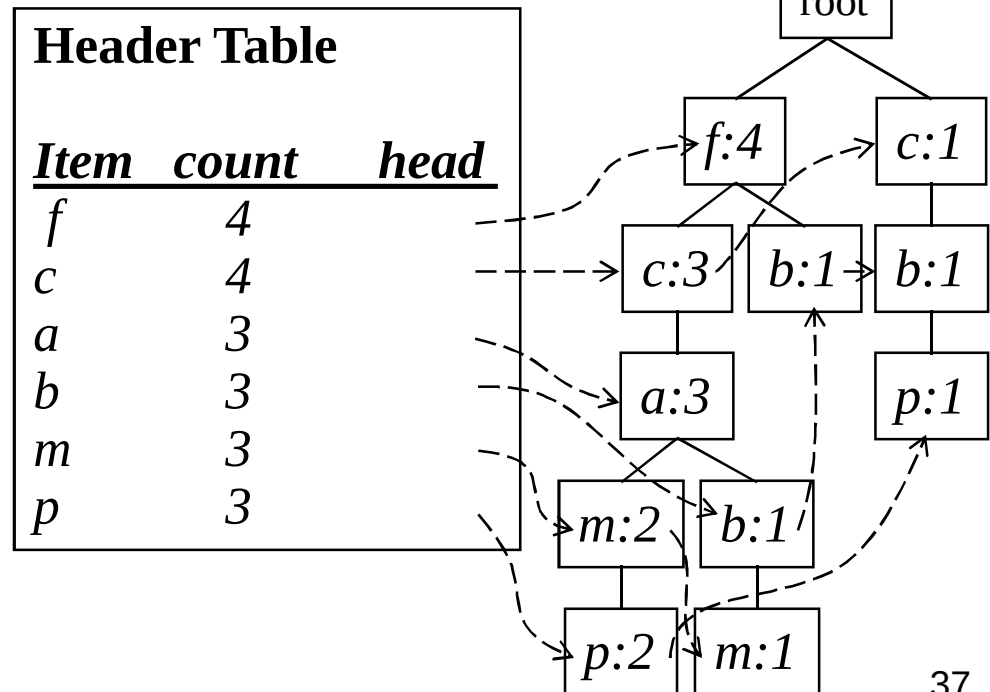
Construct FP-tree from a Transaction DB

<i>TID</i>	<i>Items bought</i>	<i>(ordered) frequent items</i>
100	{f, a, c, d, g, i, m, p}	{f, c, a, m, p}
200	{a, b, c, f, l, m, o}	{f, c, a, b, m}
300	{b, f, h, j, o}	{f, b}
400	{b, c, k, s, p}	{c, b, p}
500	{a, f, c, e, l, p, m, n}	{f, c, a, m, p}

min_support = 0.5
minimum support count = 3

Steps:

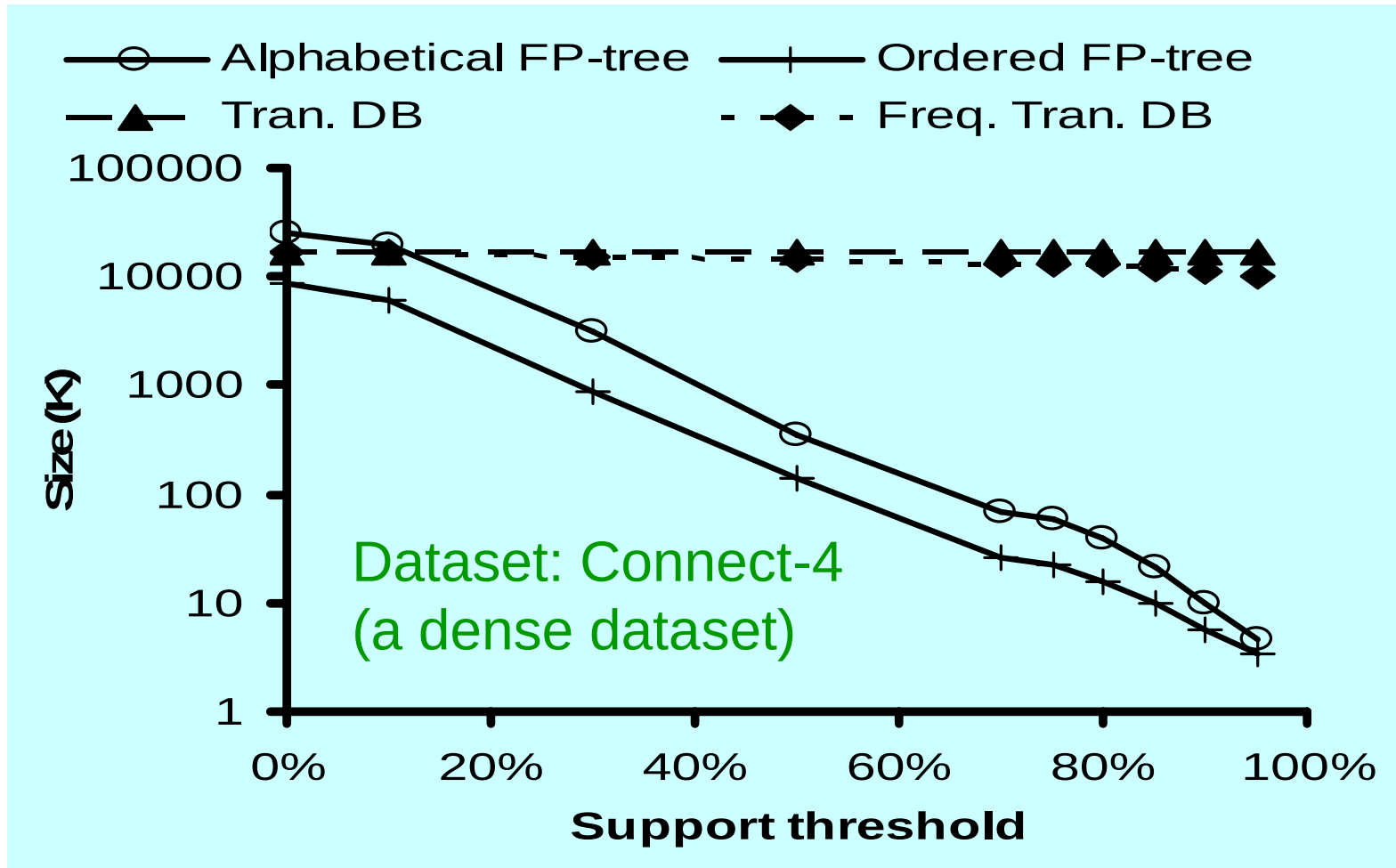
1. Scan DB once, find frequent 1-itemset (single item pattern)
2. Scan DB again, construct FP-tree
 1. Remove infrequent items
 2. Order frequent items in frequency descending order
 3. Put the remaining items in the tree



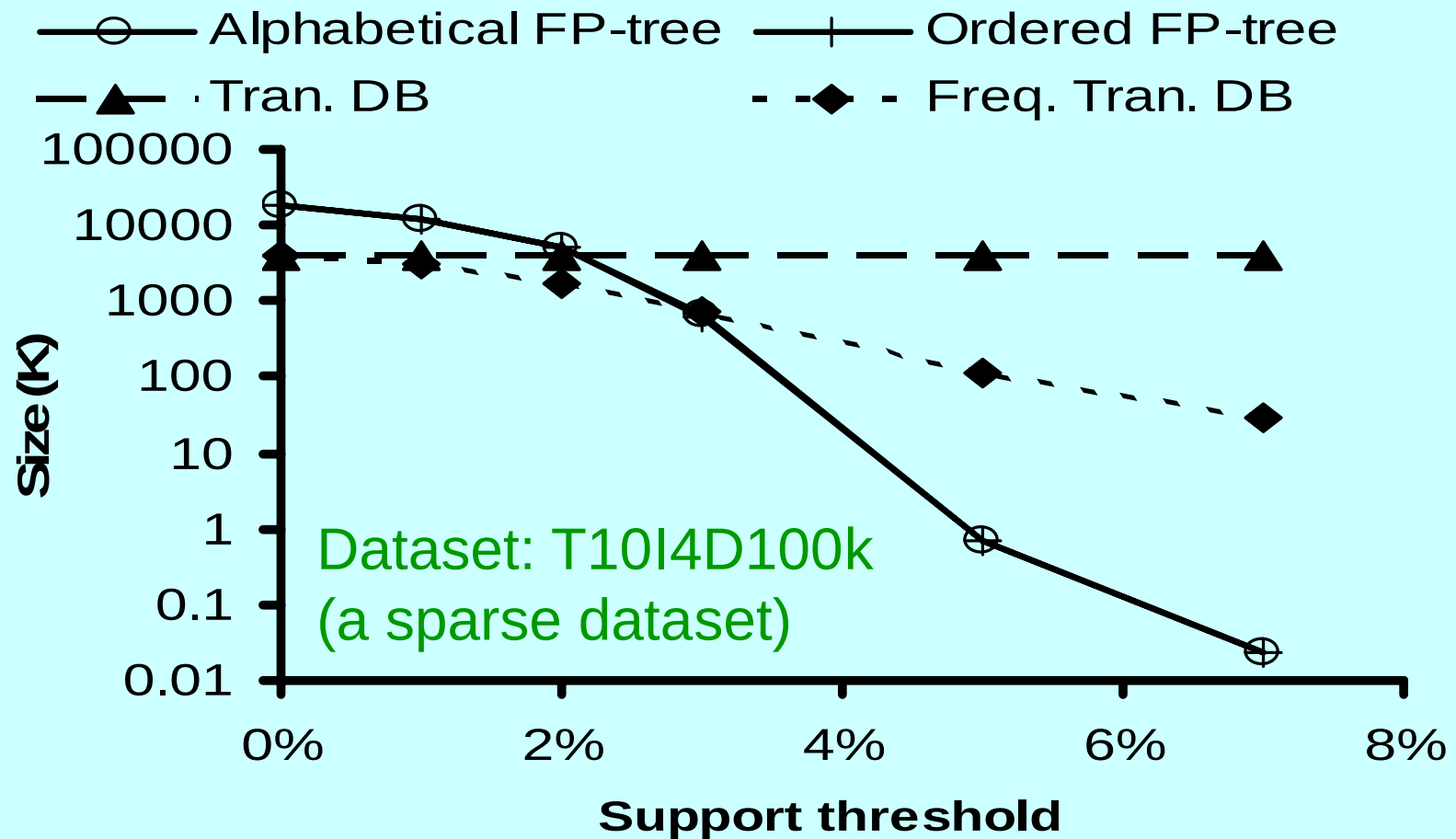
Benefits of the FP-tree Structure

- ▶ Completeness:
 - ▶ map each transaction into a path in the tree
 - ▶ preserves complete information for frequent pattern mining
 - ▶ no need to scan the database any more
- ▶ Compactness
 - ▶ reduce irrelevant information—infrequent items are gone
 - ▶ A path can store one or more transactions
 - ▶ Items in frequency descending order (*f-list*):
 - ▶ more frequent items are more likely to be shared
 - ▶ Not larger than the original database (not counting node-links and the count fields)

How Effective Is FP-tree?



Compressing Sparse Dataset



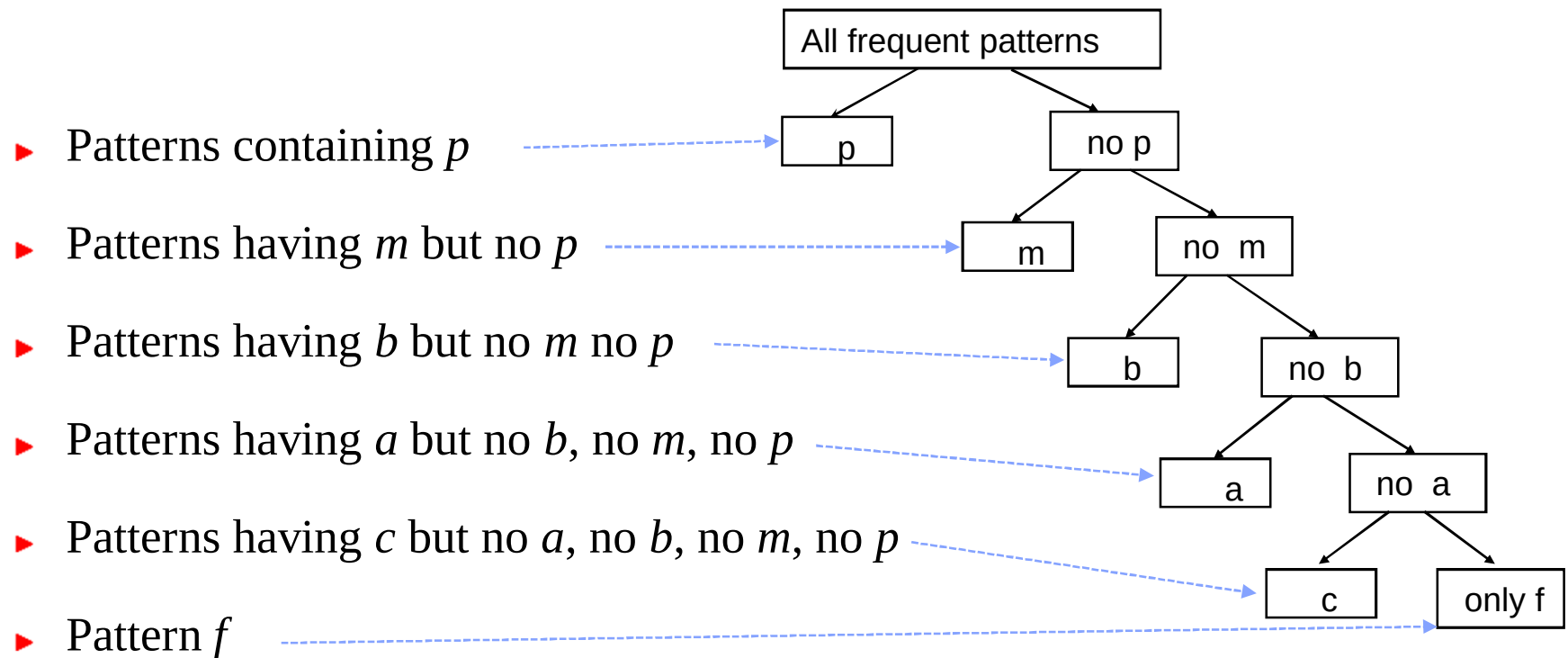
Mining Frequent Patterns from FP-tree

(Frequent pattern = frequent itemset)

- ▶ General idea:
 - ▶ Divide-and-conquer
 - ▶ Recursively
 - ▶ partition the set of frequent patterns
 - ▶ find frequent patterns in each partition by
 - ▶ building *conditional pattern base* and *conditional FP-tree* for each local frequent item in the partition

Partition the Set of Frequent Patterns

- ▶ Partition the set of frequent patterns into subsets according to **f-list: f-c-a-b-m-p** (the list of freq. items in frequency- descending order):



- ▶ The partitioning is complete and without any overlap

Find Frequent Patterns Having Item “p”

- ▶ Only transactions containing p are needed
- ▶ Form *p*-conditional pattern base (*p*-projected database) $TDB|_p$ → Contains transactions containing p
 - ▶ Starting at entry p of header table
 - ▶ Follow the side-link of frequent item p
 - ▶ Accumulate all transformed prefix paths of p

p-conditional pattern base $TDB|_p$

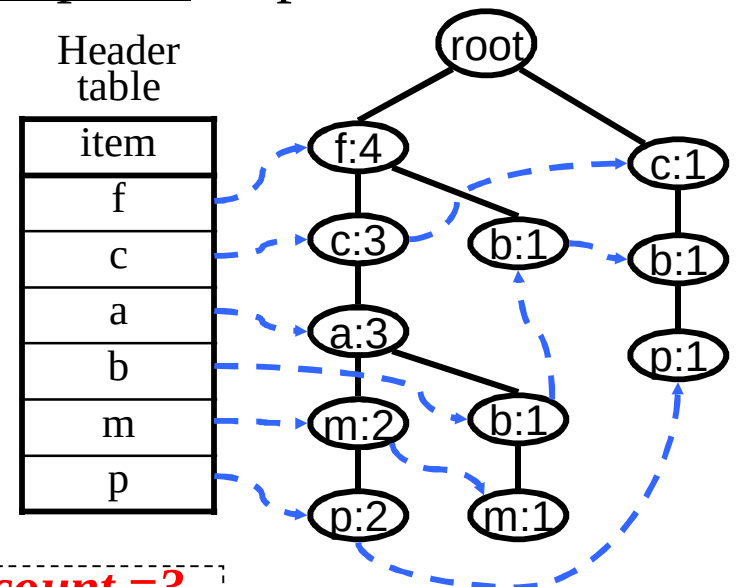
fcam: 2

cb: 1

Local frequent item: c:3

Frequent patterns containing p

cp: 3



minimum support count = 3

Find Frequent Patterns Having Item m But No p

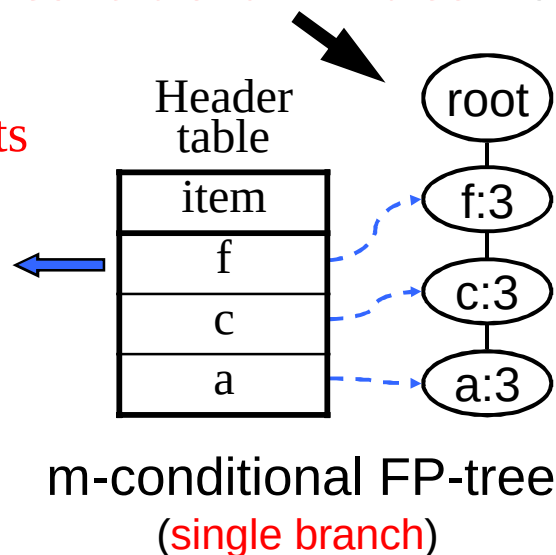
- ▶ Form m-conditional pattern base (m-projected database) TDB|m
 - ▶ Item p is excluded (by looking at only the prefix paths of m)
 - ▶ TDB|m contains **fca:2, fcab:1**
- ▶ Recursively apply FP-growth to find freq. patterns from TDB|m
 - ▶ Local frequent items: f, c, a
 - ▶ After removing local infrequent item: **fca:2, fca:1**
 - ▶ Build **m-conditional FP-tree** from TDB|m

Non-empty subsets
of {f, c, a}:

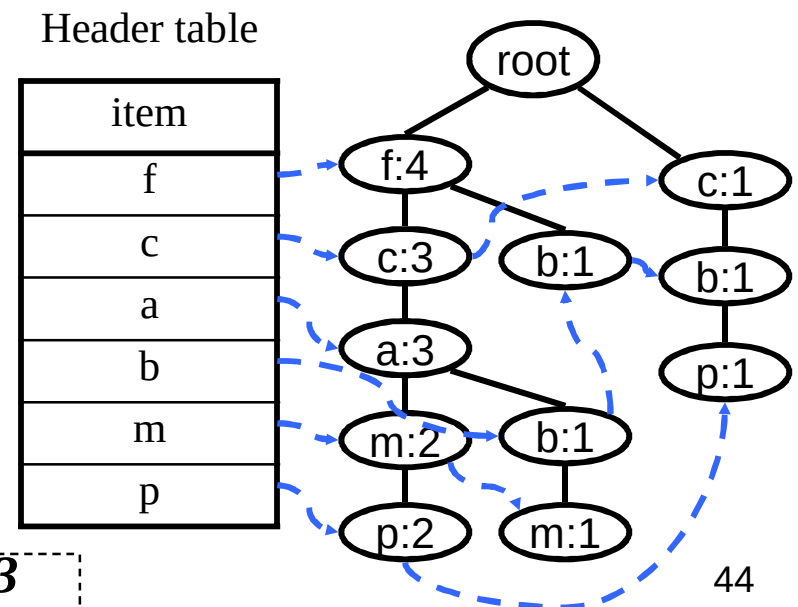
f, c, a
fc, fa, ca
fca

Frequent patterns
having m:

fm, cm, am
fcm, fam, cam
fcam



minimum support count = 3



Find Frequent Patterns Having Item m But No p (*more complex situation*)

▶ Assume m-conditional pattern base is: **fca:3, fb:3**

▶ Local frequent items: f:6, c:3, a:3, b:3

▶ Build m-conditional FP-tree

▶ First generate:

▶ fm: 6, cm: 3, am:3, bm:3

▶ To learn longer patterns containing m

▶ Further partition frequent patterns containing m (but no p) into

▶ Patterns containing b

▶ Patterns containing a but no b

▶ Patterns containing c but no b or a

▶ Patterns containing only f (i.e. fm)

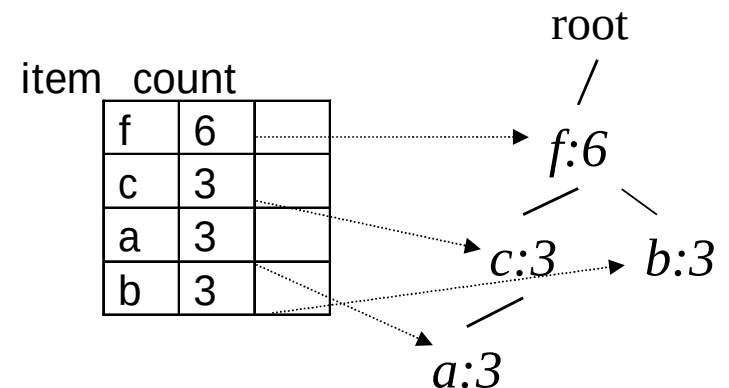
▶ Compute *ym*-conditional pattern bases:

ym conditional pattern base

bm *f:3*

am *fc:3*

cm *f:3*



Find Frequent Patterns Having Item m But No p (*more complex situation*)

- ▶ Having *ym*-conditional pattern bases:

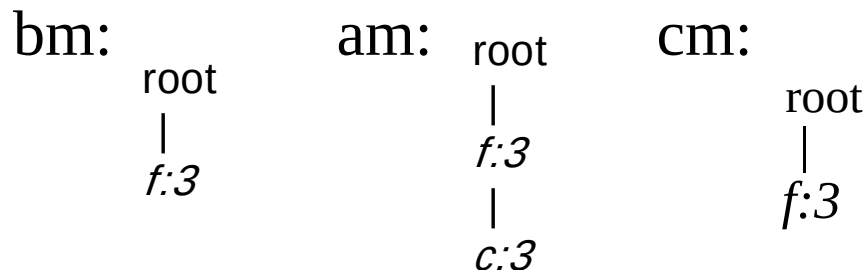
ym conditional pattern base

bm *f:3*

am *fc:3*

cm *f:3*

- ▶ Built *ym*-conditional FP-trees



- ▶ General frequent patterns with suffix *ym*:

fbm, *fam*, *cam*, *fcam*, *fc**m*

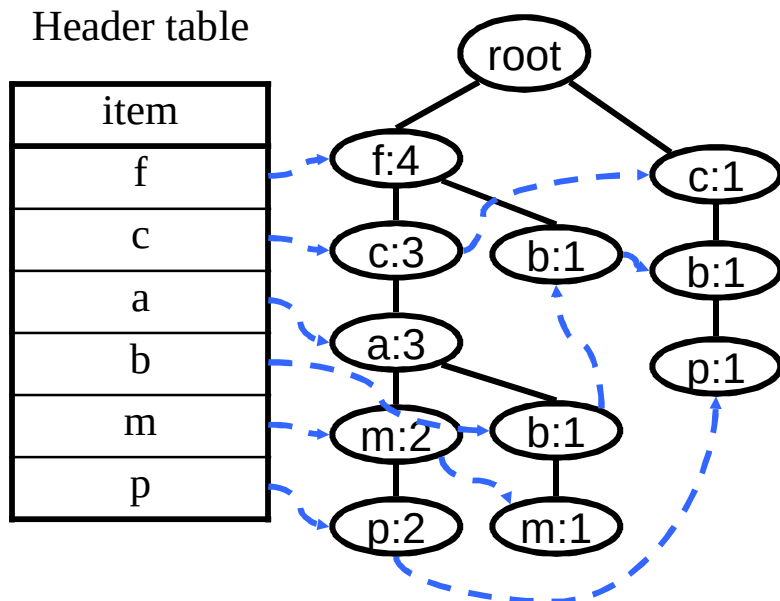
Major Steps to Mine FP-tree

For each item in the FP-tree

1. Construct *conditional pattern base*
2. Construct *conditional FP-tree* from the conditional pattern-base
3. Generate frequent patterns from the conditional FP-tree
 - ▶ If the conditional FP-tree contains a single path, simply enumerate all the subsets of the itemset on the path (*Recursion stopping criterion*)
 - ▶ Otherwise, ***recursively*** mine the conditional FP-tree and grow frequent patterns obtained so far

Step 1: From FP-tree to Conditional Pattern Base

- ▶ Starting at the frequent header table in the FP-tree
- ▶ Traverse the FP-tree by following the link of each frequent item
- ▶ Accumulate all prefix paths of that item to form a *conditional pattern base*



Conditional pattern bases

<u><i>item</i></u>	<u><i>cond. pattern base</i></u>
<i>c</i>	<i>f:3</i>
<i>a</i>	<i>fc:3</i>
<i>b</i>	<i>fca:1, f:1, c:1</i>
<i>m</i>	<i>fca:2, fcab:1</i>
<i>p</i>	<i>fcam:2, cb:1</i>

Step 2: Construct Conditional FP-tree

- ▶ For each pattern-base
 - ▶ Accumulate the count for each item in the base to find local frequent items
 - ▶ Remove locally infrequent items
 - ▶ Construct conditional FP-tree for the frequent items of the pattern base

p-conditional pattern

base:

fcam:2, cb:1



Taken out infrequent items

c:2, c:1



root



c:3

p-conditional FP-tree

m-conditional pattern

base:

fca:2, fcab:1



Taken out infrequent items

fca:2, fca:1



root



f:3



c:3



a:3

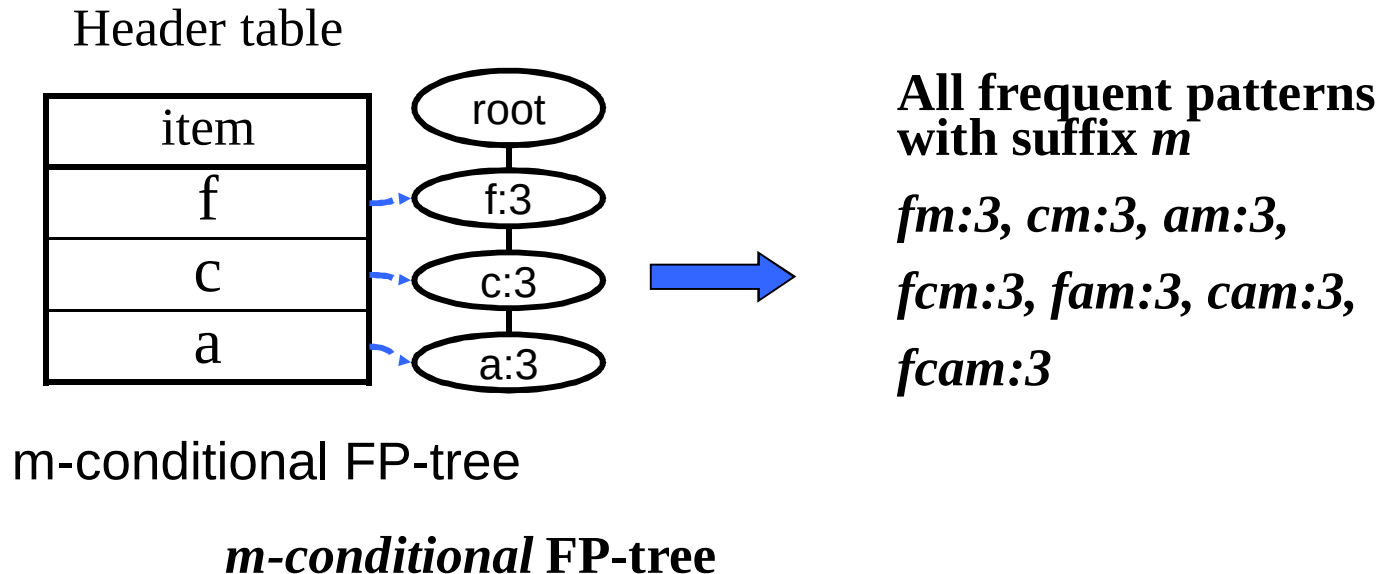
m-conditional FP-tree

Conditional Pattern-Bases and Conditional FP-trees

Item	Conditional pattern-base	Conditional FP-tree
p	$\{(fcam:2), (cb:1)\}$	$\{(c:3)\} p$
m	$\{(fca:2), (fcab:1)\}$	$\{(f:3, c:3, a:3)\} m$
b	$\{(fca:1), (f:1), (c:1)\}$	Empty
a	$\{(fc:3)\}$	$\{(f:3, c:3)\} a$
c	$\{(f:3)\}$	$\{(f:3)\} c$

Step 3: Generate Frequent Patterns from Conditional FP-tree

- ▶ If an x -conditional FP-tree has a single path P
 - ▶ The complete set of frequent patterns with suffix x can be generated by enumeration of all the combinations of items in P

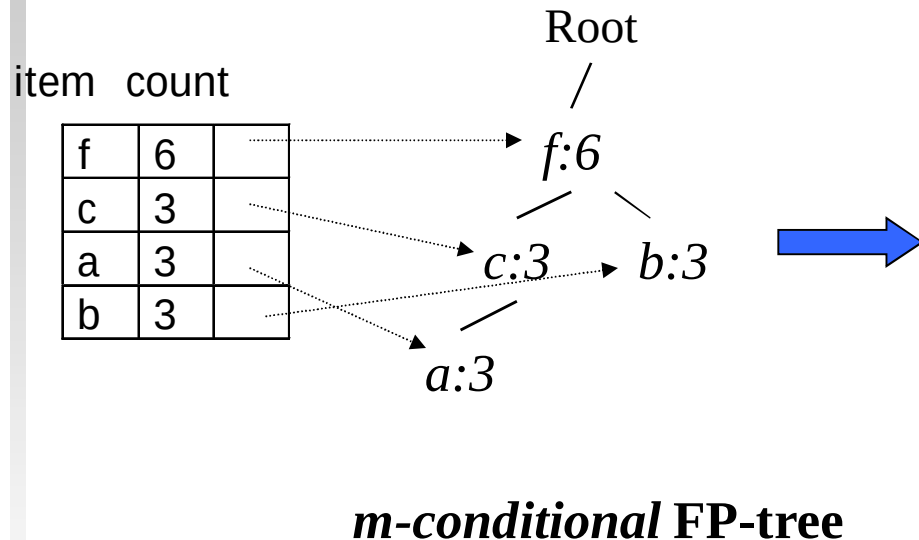


Step 3: Generate Frequent Patterns from Conditional FP-tree (*Contd.*)

- ▶ **If an x -conditional FP-tree has more than one path**
 - ▶ For each item y that appears in x -conditional FP-tree
 - ▶ Generate pattern yx with support = the support of y in x -conditional FP-tree.
 - ▶ Construct yx -conditional pattern base and then yx -conditional FP-tree to generate frequent patterns with suffix yx (a recursive procedure).

Step 3: Generate Frequent Patterns from Conditional FP-tree (*Contd.*)

- Suppose m -conditional FP-tree is



- Generate frequent 2-itemsets having m :
 $fm:6, cm:3, am:3, bm:3$

- Compute ym -conditional pattern bases:

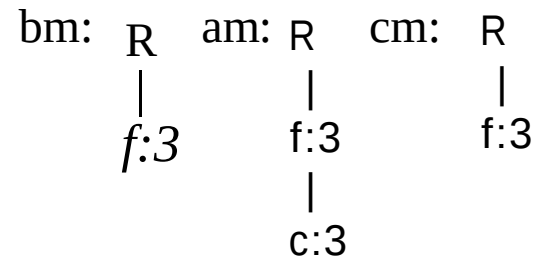
ym conditional pattern base

bm $f:3$

am $fc:3$

cm $f:3$

- Built ym -conditional FP-trees



- General frequent patterns with suffix ym :
 $fbm:3, fam:3, cam:3, fcam:3, fcm:3$

FP-Growth Algorithm

- ▶ Input: *FP-tree* (a FP-tree built by scanning DB)
- ▶ Output: the complete set of frequent patterns
- ▶ Method: call `FP-growth(FP-tree, null)`
- ▶ Procedure `FP-growth(A_conditional_FP_Tree, A)`
 - ▶ if Tree contains a single path *P*
 - ▶ for each combination (denoted as *B*) of the nodes in the path *P* do
 - ▶ generate pattern *BA* with support = minimum support of nodes in *B*
 - ▶ else for each item a_i in the header table of *Tree* do
 - ▶ generate pattern $B=a_iA$ with support = the support of a_i
 - ▶ construct *B*'s conditional pattern base and then *B*'s conditional FP-tree $Tree_B$;
 - ▶ if $Tree_B$ is not empty,
 - ▶ call **FP-growth**($Tree_B$, *B*)

Exercise

- ▶ A transaction DB:

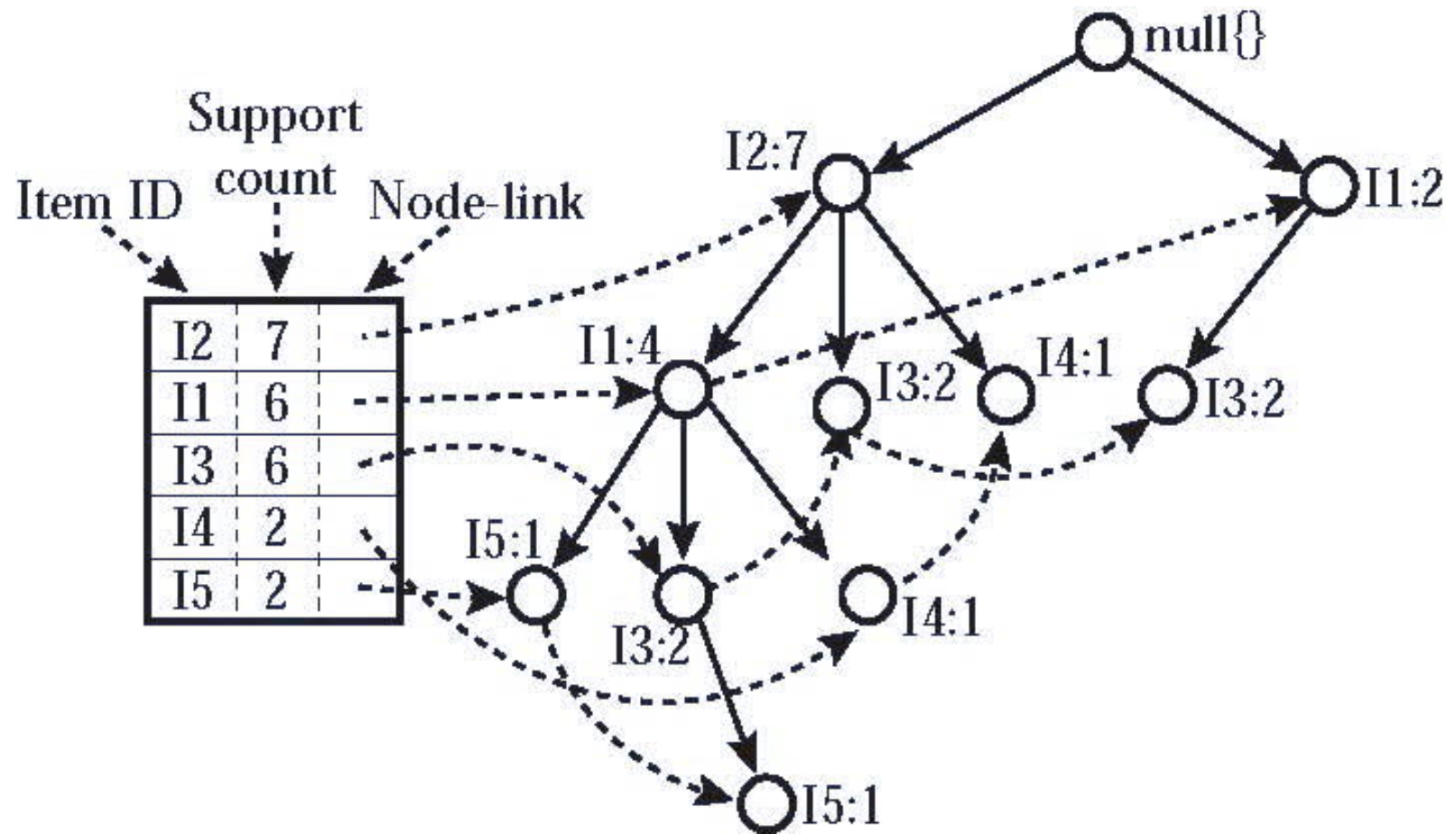
TID	items
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

- ▶ Support counts
for single items:

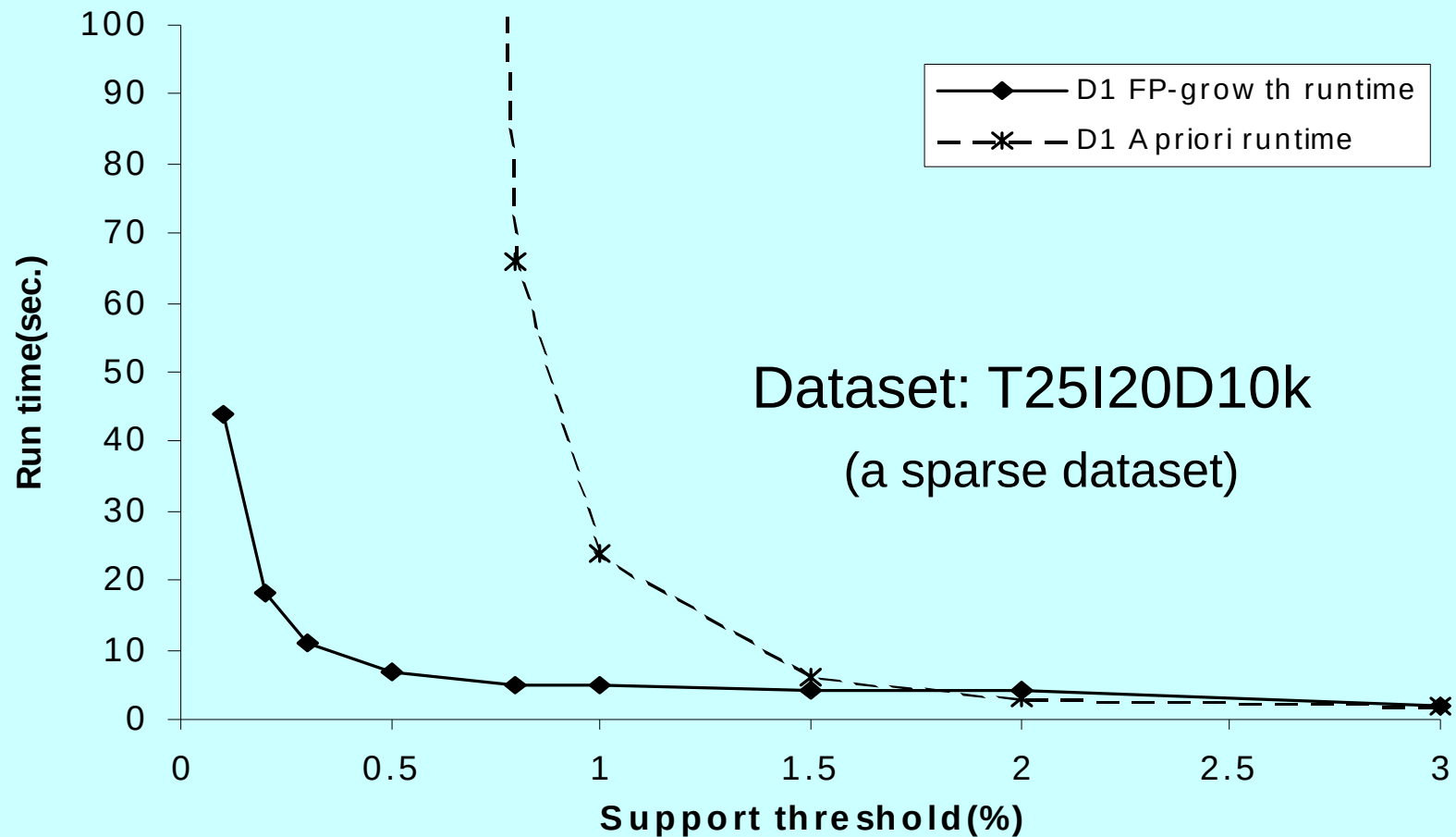
Item	Sup. count
{I1}	6
{I2}	7
{I3}	6
{I4}	2
{I5}	2

- ▶ Find all frequent patterns with minimum support count =2.

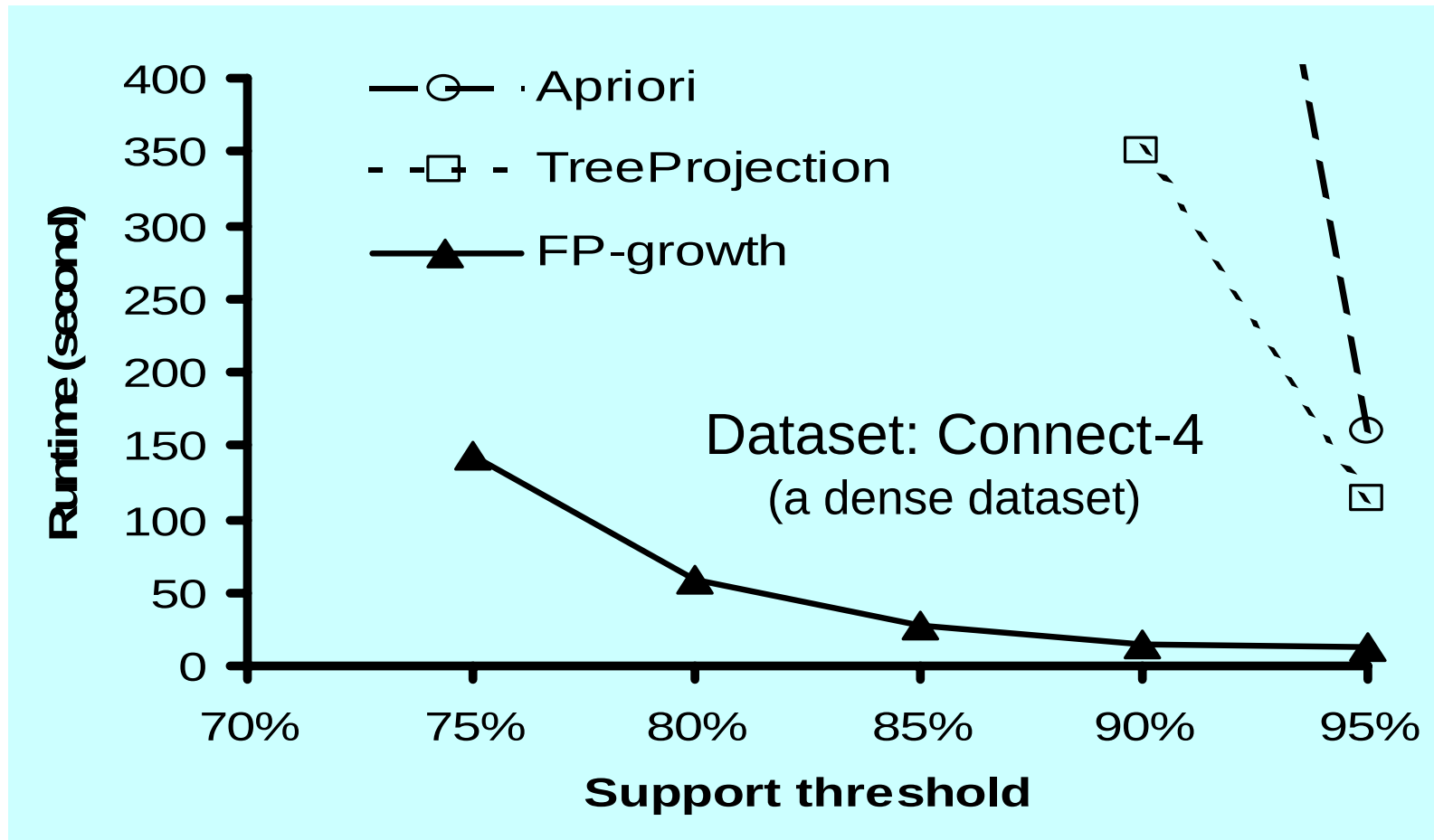
FP-tree



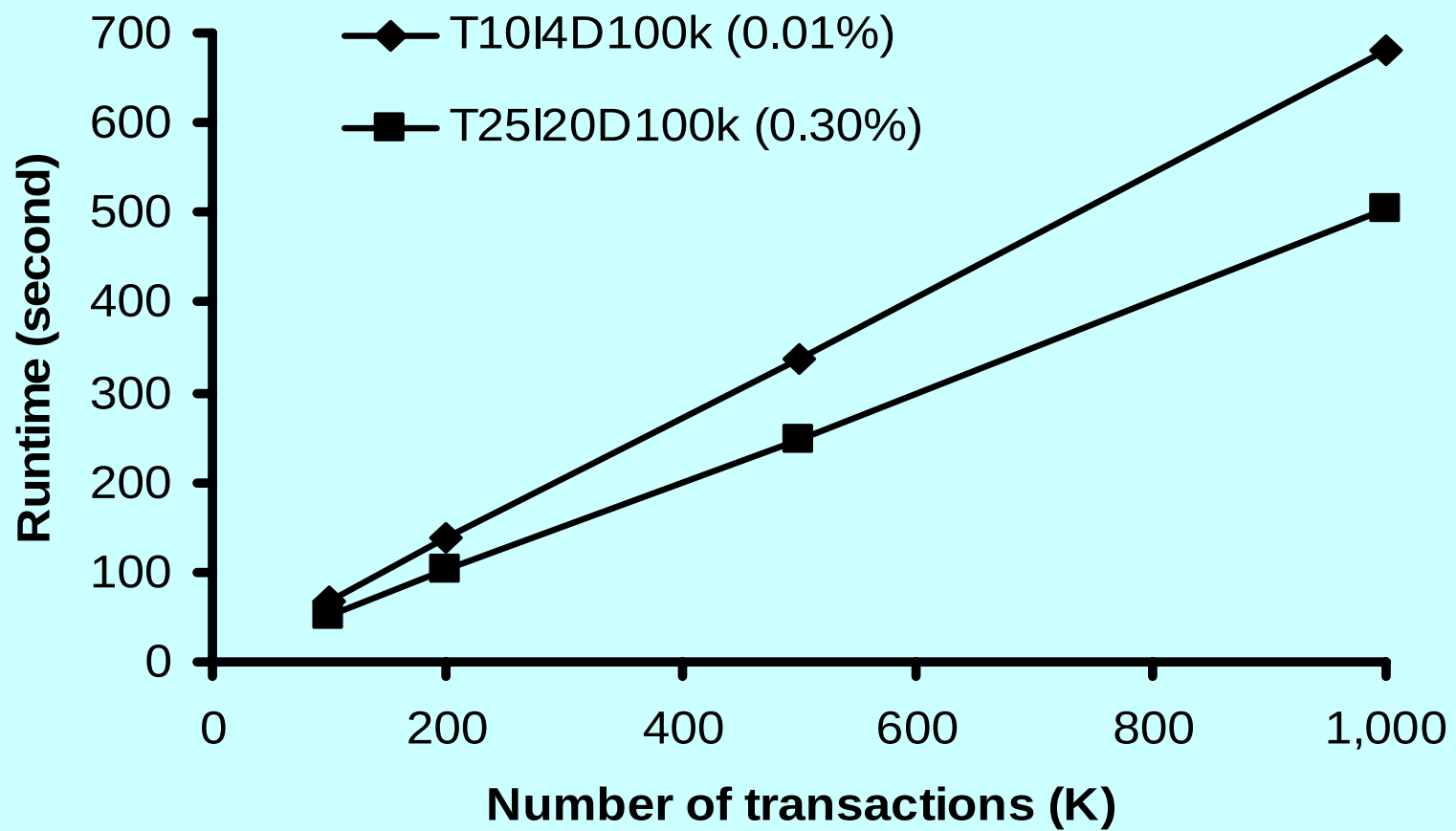
FP-growth vs. Apriori



Mining Very Dense Dataset



Scalability of FP-growth



Why Is FP-growth Efficient?

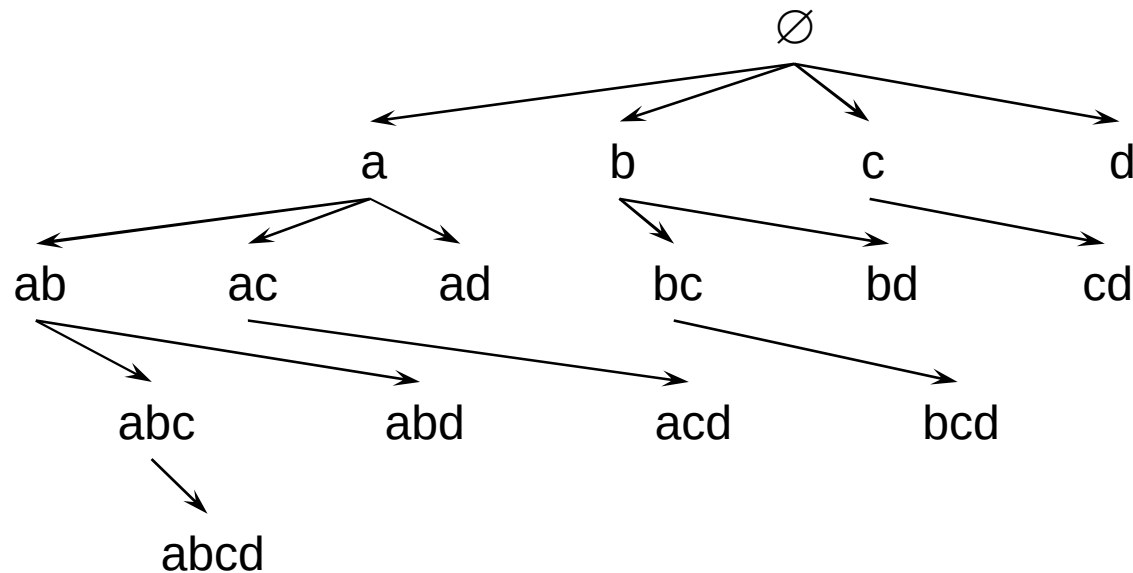
- ▶ Divide-and-conquer strategy
 - ▶ Decompose both the mining task and DB
 - ▶ Lead to focused search of smaller databases
- ▶ No candidate generation nor candidate test
- ▶ Database compression using FP-tree
 - ▶ No repeated scan of entire database
- ▶ Basic operations:
 - ▶ counting local freq items and building FP-tree, no pattern search nor pattern matching

Major Costs in FP-growth

- ▶ Building FP-trees
 - ▶ A stack of FP-trees
- ▶ Redundant information is stored in a stack of FP-trees.
- ▶ Can we avoid the redundancy?
 - ▶ H-mine (another algorithm by Pei and Han)
 - ▶ Based on a different data structure.
 - ▶ Less memory usage
 - ▶ Better for sparse datasets
 - ▶ There is also other improvement to FP-growth based on the FP-tree structure.

Compare FP-growth to Apriori

- ▶ Search space for DB with 4 items:



- ▶ Apriori: breadth-first
- ▶ FP-growth: depth-first

What if FP-tree cannot fit in memory?

(Partition to reduce # of disk scans)

- ▶ Partition DB
 - ▶ Each partition is held in main memory
- ▶ Any itemset that is potentially frequent in DB must be frequent in at least one of the partitions of DB (can be proved)
 - ▶ Scan 1: partition database and for each partition find local frequent patterns
 - ▶ Scan 2: consolidate global frequent patterns
- ▶ A. Savasere, E. Omiecinski, and S. Navathe. An efficient algorithm for mining association in large databases. In *VLDB'95*